

PBA: 2. R Programming Basics

Fall 2025

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Roadmap

1. Working in Plain Text
2. Let's Take A TouR
3. Using Rmarkdown
4. Getting R Bearings
 - Types of Programming Errors
 - Operators in R
5. Inferential Statistics Redux

1/ Working in Plain Text

The Two Computer Revolutions



The Frontier of Computing

- Touch-based interfaces
- Single app at a time
- Little multitasking between apps
- Hides the file system



Where Statistical Computing Lives

- Windows and pointers
- Multi-tasking, multiple windows
- Works heavily with the file system
- Underneath it's UNIX and the command line

Plain-Text Tools for Data Analysis



- Often free, open-sourced, and powerful
 - e.g., R, RMarkdown, Git (for version control)
- Large, friendly communities around them
- Tons of resources
- But... far from the touch-based paradigm of modern computing

⇒ **So why use them?**

The process of data science is intrinsically messy

Office vs. Engineering Model of Computing

What's real in the project? How are changes managed?

In the Office Model

- Formatted documents are real
- Intermediate outputs copy/pasted into documents
- Changes are tracked inside files
- Final output is the file you're working on (e.g., Word or PDF)

In the Engineering Model

- Plain-text files are real
- Intermediate outputs are produced via code
- Changes are tracked outside files
- Final outputs are assembled programmatically and converted to the desired format

Pros and Cons to Each Approach

- Office model:
 - Everyone knows Word, Excel, Google Docs.
 - "Track changes" is powerful and easy.
 - Wait, how did I make this figure?
 - Which version of my code made this table?
 - Jaewon_Yoo_report_final_submitted_edits_FINAL_v2.docx
- Engineering model:
 - Plain text is universally portable.
 - Push button, recreate analysis.
 - Why won't R just do what I want!
 - Version control is a pain.
 - Object of type `'closure'` is not subsettable.

We'll tend toward the Engineering model because it's better suited to keep the mess in check.

2/ Let's Take A Tour

R versus RStudio

```
R
R version 4.2.1 (2022-06-23) -- "Funny-Looking Kid"
Copyright (C) 2022 The R Foundation for Statistical Computing
Platform: aarch64-apple-darwin20 (64-bit)

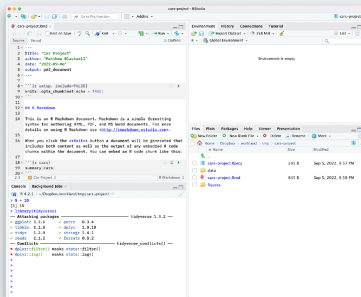
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

>
```



An Overview

Write notes and codes in R Markdown or in an R Script

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

When you click the Render button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
(r)
1 + 1
```

You can add options to executable code like this

```
(r)
# echo: false
2 + 2
```

The `echo: false` option disables the printing of code (only output is displayed).

Environment is empty

Console

```
R 4.2.3 -- (R)

R version 4.2.3 (2023-03-15) -- "Shortstop Beagle"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: x86_64-apple-darwin20 (64-bit)

It is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

Warning: macOS is reporting that you have not yet agreed to the Xcode license.
This can occur if Xcode has been updated or reinstalled (e.g. as part of a macOS update).
Some features (e.g. Git / SVN) may be disabled.
```

An Overview

The screenshot displays the RStudio interface with a Quarto document open. The document content includes a YAML header, a title, author information, and a section on running code. The console at the bottom shows the R version and startup information.

Quarto Document Content:

```
---
title: "PMA EXAMPLE"
author: "Samuel Teo"
format: html
editor: visual
---
```

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

When you click the Render button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
(r)
1 + 1
```

You can add options to executable code like this

```
(r)
# echo: false
2 + 2
```

The `echo: false` option disables the printing of code (only output is displayed).

Console Output:

```
R version 4.2.3 (2023-03-15) -- "Shortstop Beagle"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: x86_64-apple-darwin20 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

Warning: macOS is reporting that you have not yet agreed to the Xcode license.
This can occur if Xcode has been updated or reinstalled (e.g. as part of a macOS update).
Some features (e.g. Git / SVN) may be disabled.
```

Console: Run code, send code here, inspect output

An Overview

The screenshot displays the RStudio IDE interface. The main editor window shows a Quarto document with a YAML header, a title, author, format, and editor. Below the header, the text "Quarto" is followed by a paragraph explaining its purpose. A section titled "Running Code" describes how to generate content from executable code. Two code blocks are shown: one with a simple R expression and another with a comment. The environment pane on the right shows an empty environment. The console at the bottom displays the R version and copyright information, along with a warning about the Xcode license.

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

When you click the Render button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
(r)
1 + 1
```

You can add options to executable code like this

```
(r)
# echo: false
2 + 2
```

The `echo: false` option disables the printing of code (only output is displayed).

Environment

Environment is empty

Console

```
R 4.2.3 -- (2023-03-15) -- "Shortstop Beagle"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: mach64-apple-darwin20 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

Warning: macOS is reporting that you have not yet agreed to the Xcode license.
This can occur if Xcode has been updated or reinstalled (e.g. as part of a macOS update).
Some features (e.g. Git / SVN) may be disabled.
```

A CloseR Look

- Code that you can type and run:

```
1 ## Any R code that begins with the # character is a comment
2 ## Comments are ignored by R
3 > my_numbers <- c(4, 8, 15, 16, 23, 42) # Anything after # is also a comment
```

- Output from code are prefixed with element number(s)/counter in brackets:

```
1 > my_numbers ## we just created this.
2 [1] 4 8 15 16 23 42
3
4 > letters ## this is built into R
5 [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l"
6 [13] "m" "n" "o" "p" "q" "r" "s" "t" "u" "v" "w" "x"
7 [25] "y" "z"
8
9 > pi ## this is also built in
10 [1] 3.141593
```

We Do Things in R with Functions

- Functions take in objects, perform actions, and return outputs:

```
1 mean(x = my_numbers)
2 [1] 18
```

- where x is the argument name,
- my_number is what we are passing to that argument.

- If you omit the argument name, R will assume the default order:

```
1 mean(my_numbers)
2 [1] 18
```

Getting Help with R

- How do we know the default argument order? Examine the (built-in) help files:

```
1 help(mean)
2 ?mean ## shorter
```

- Sometimes inscrutable, so it helps to look else where:
 - Google, StackOverflow, Twitter, RStudio Community
 - Ask on Canvas discussion board!
 - Come talk to me & TAs for help
- Get help **early** before becoming too frustrated!
 - Easy to overlook small issues like missing commas, etc.

Functions in R 'Live' in Packages

- Packages are bundles of functions written by other users that we can use:

```
1 install.packages("ggplot2")
```

- Install packages using `install.packages()` to have them set up on your machine.

```
1 library(ggplot2) ## or require()
```

- Load them into your R session with `library()` or `require()`

⇒ We will now be able to use any function provided by `ggplot2`

Functions in R ‘Live’ in Packages (Cont)

- We can also use the `packageName::` prefix to access package functions without loading it:

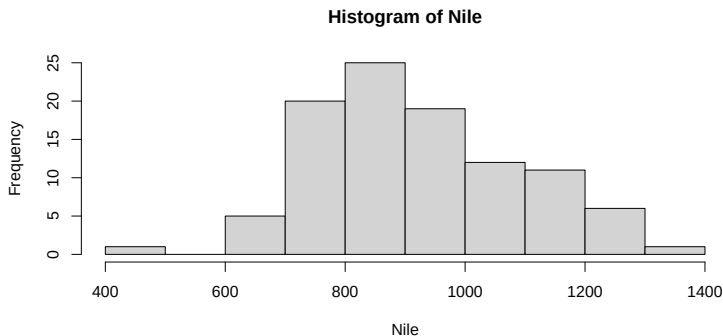
```
1 knitr::kable(head(mtcars))
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.62	16.46	0	1	4	4
Mazda RX4	21.0	6	160	110	3.90	2.88	17.02	0	1	4	4
Wag											
Datsun 710	22.8	4	108	93	3.85	2.32	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.22	19.44	1	0	3	1
Hornet Sport- about	18.7	8	360	175	3.15	3.44	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.46	20.22	1	0	3	1

What can we do with R?

Examine Descriptive Statistics & Visualize Data

```
1 > data(Nile)
2 > mean(Nile)
3 [1] 919.35
4 > sd(Nile)
5 [1] 169.2275
6 > hist(Nile)
```



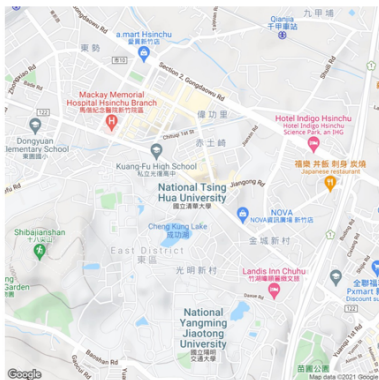
Basic math: use it as a calculator!

```
1 > 1 + 2 + 3
2 [1] 6
3 > 3 * 7 * 2
4 [1] 42
5 > 4 / 2
6 [1] 2
7 > 4 / 3
8 [1] 1.333333
9 > 4 * 6 + 5
10 [1] 29
11 > (4 * 6) + 5
12 [1] 29
13 > 4 * (6 + 5)
14 [1] 44
15 > 4 > 5
16 [1] FALSE
17 > 4 <= 5
18 [1] TRUE
19 > # comments
```

Spatial Analysis and Geoprocessing

- Work w/ geographical data and present interesting patterns in a readily interpretable way directly on a map:

```
1 > install.packages("ggmap")
2 > require(ggmap)
3 > register_google(googleAPIkey)
4
5 > ggmap(get_googlemap(center = c("national
  ↳ tsing hua university"), zoom = 15), extent
  ↳ = "device")
```



Work with Unstructured Data

- Draw word clouds or run natural language processing (NLP):

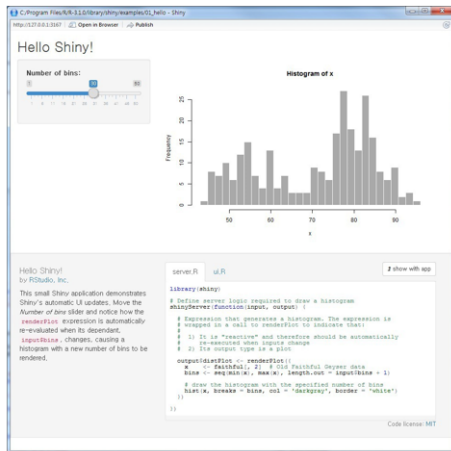
```
1 > install.packages("wordcloud")
2 > install.packages("tm")
3 > library(wordcloud); library(tm)
4
5 > wordcloud("R is a language and environment for
↳ statistical computing and graphics. It is a GNU
↳ project which is similar to the S language and
↳ environment which was developed at Bell Laboratories
↳ (formerly AT&T, now Lucent Technologies) by John
↳ Chambers and colleagues. R can be considered as a
↳ different implementation of S. There are some
↳ important differences, but much code written for S
↳ runs unaltered under R.")
```



Better Delivery with Interactive Reports

- Produce interactive reports for effective communication of results:

```
1 > install.packages("shiny")  
2 > library(shiny)  
3 > runExample("01_hello")
```



3/ Using Rmarkdown

The Acts of Coding

```
library(ggplot2)
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```

Figure 1. Writing code

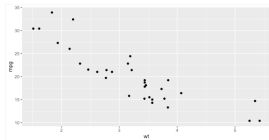


Figure 2. Looking at output

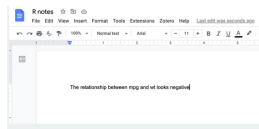
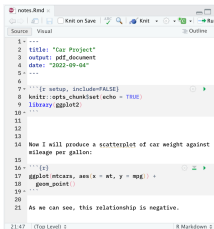


Figure 3. Taking notes

How to do all of these efficiently?

Rmarkdown Files to the Rescue



```
1 ---
2 title: "Car Project"
3 output: pdf_document
4 date: "2022-09-04"
5 ---
6
7 ```{r setup, include=FALSE}
8 knitr::opts_chunk$set(echo = TRUE)
9 library(ggplot2)
10 ```
11
12
13 Now I will produce a scatterplot of car weight against
14 mileage per gallon:
15
16 ```{r}
17 ggplot(mtcars, aes(x = wt, y = mpg)) +
18   geom_point()
19 ```
20
21 As we can see, this relationship is negative.
```

Fig.: Rmarkdown file

Keep code and
notes together in
plain text

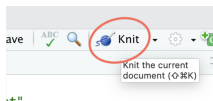


Fig.: Knit in R

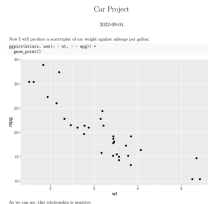


Fig.: PDF output

Produce
nice-looking
outputs in
different formats

Markdown: Formatting in Plain Text

Non-code text in Rmd files is plain text with formatting instructions

syntax

Plain text
End a line with two spaces to start a new paragraph.
italics and _italics_
bold and __bold__
superscript^2^
~~strikethrough~~
[link](www.rstudio.com)

Header 1

Header 2

Header 3

Header 4

Header 5

Header 6

endash: --

endash: ---

ellipsis: ...

inline equation: $A = \pi r^2$

image:

horizontal rule (or slide break):

> block quote

* unordered list

* item 2
+ sub-item 1
+ sub-item 2

1. ordered list

2. item 2
+ sub-item 1
+ sub-item 2

becomes

Plain text
End a line with two spaces to start a new paragraph.
italics and *italics*
bold and **bold**
^{superscript²}
~~strikethrough~~
[link](#)

Header 1

Header 2

Header 3

Header 4

Header 5

Header 6

endash: --

endash: ---

ellipsis: ...

inline equation: $A = \pi r^2$

image: 

horizontal rule (or slide break):

block quote

* unordered list

* item 2
+ sub-item 1
+ sub-item 2

1. ordered list

2. item 2
+ sub-item 1
+ sub-item 2


```
1 ---
2 title: "Car Project"
3 author: "Jaewon Yoo"
4 date: "2024-08-01"
5 output: pdf_document
6 ---
```

Header contains metadata
and sets options about the
whole document

```
7
8 ```{r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE)
10 ```
```

Code chunk

Plain text with markdown
formatting

```
11
12 ## R Markdown
```

```
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word
documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.
```

```
15
16 When you click the Knit button a document will be generated that includes both content as well as the output of
any embedded R code chunks within the document. You can embed an R code chunk like this:
```

```
17
18 ```{r cars}
19 summary(cars)
20 ```
```

Chunks can have
names and options

Can "play" chunks
interactively

```
21
22 ## Including Plots
```

```
23
24 You can also embed plots, for example:
```

```
25
26 ```{r pressure, echo=FALSE}
27 plot(pressure)
28 ```
```

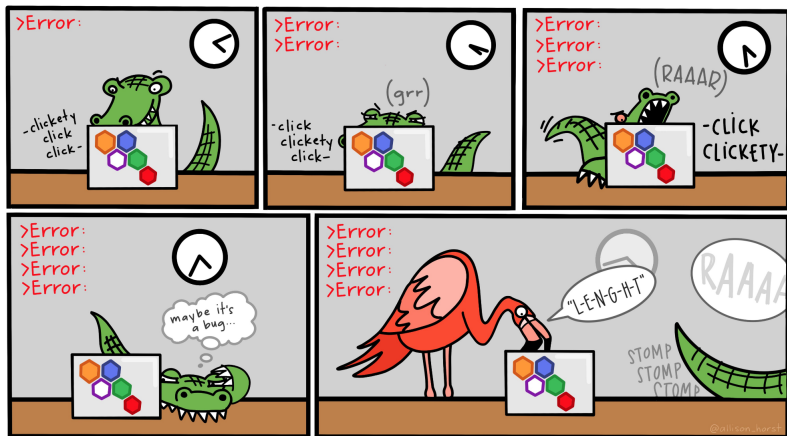
Code chunks replaced
with output when Knitted

```
29
30 Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated
the plot.
```

4/ Getting R Bearings

Try to Type Your Code by Hand!

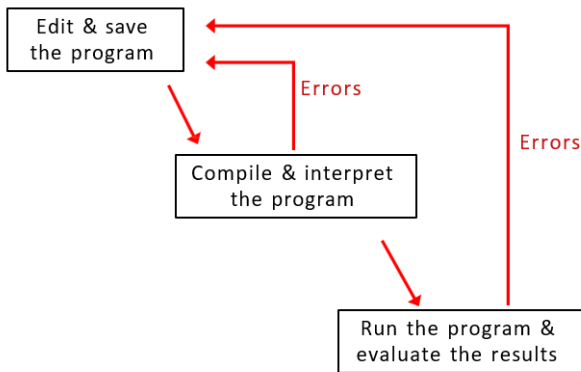
Typing Speeds Up the Try-Fail Cycle



Physically typing the code is best way to familiarize yourself with R and the try-fail-try-fail-try-succeed cycle!

Programming Procedure

- Programming is an iterative process (try-fail-try-fail-try-succeed):



- Compilation: a process of converting high-level (computer) language into an executable low-level language/machine codes.

Programming Errors

- Three different types of errors consist of:
 1. **Compilation errors**
 2. **Runtime errors**
 3. **Logic errors**
- **Compilation errors** occur when there are syntactic (i.e., syntax) violations such that the compiler is not able to understand your code.
- **Runtime errors** take place while the program is running after being successfully compiled.
- **Logic errors** is where your code is perfectly “legal” syntactically but fails to do what you intend for it to do.

Examples?

- Compilation errors:

```
1 > 3 % 4
```

Error: unexpected input in "3 % 4"

- Runtime errors:

```
1 > x<-1:50  
2 > i<-51  
3 > x[i]
```

NA

- Logic errors:

```
1 # Sum of all numbers from 1 to 100  
2 > sum(1:100)  
3 # [1]5050  
4 # Sum of all numbers that are bigger than 50 from 1 to 100  
5 > sum(1:100 > 50)  
6 # [1]50
```

Operators in R: Arithmetic Operators

- Every journey has to start somewhere...
- These operators are used to do basic arithmetic operations like addition, subtraction, multiplication, division, exponent, modulus, etc.

Operator	Operation	Examples
*	multiplication	$a*b$
/	division	a/b
+	addition	$a+b$
-	subtraction	$a-b$
^	exponentiation	a^b
%%	modulus	$a\%b$
%/%	integer division	$a\%/\%b$

Operators in R: Relational Operators

- These operators are used to compare (e.g., equals, larger than, smaller than, etc.) two objects.

Operator	Operation	Examples
==	equals	a==b
!=	not equal	a!=b
>	greater than	a>b
>=	greater than or equal to	a>=b
<	less than	a<b
<=	less than or equal to	a<=b

Operators in R: Logical Operators

- These operators are used to perform logical/Boolean operations like AND, OR, NOT, etc. on objects.

Operator	Operation	Description
!	not	negation operator, e.g., !a
	or	elementwise or operator, e.g., a b
	or	or operator which evaluates only the first element in the objects, e.g., a b
&	and	elementwise and operator, e.g., a&b
&&	and	and operator which evaluates only the first element in the objects, e.g., a&&b

Operators in R: Assignment Operators

- The following operators are used to assign/store values (or data) on to a variable/object.

Operator	Operation	Examples
->	Rightward assignment	a -> b
->>	Rightward super-assignment	a ->> b
=	Assignment (right to left)	a = b
<-	Assignment (right to left)	a <- b
<<-	Super-assignment (right to left)	a <<- b

- Rightward assignment is possible, but it's most common to assign right to left using <- or =
- Super-assignment operators to be discussed when we learn the scoping rules.

Operators in R: Indexing Operators

- The following operators can be used to select and return specific values in a variable/object.

Operator	Operation	Examples
[]	Indexing which returns elements of list class	a[b]
[[]]	Indexing which returns an object of the class contained in the list	a[[b]]
\$	Indexing single elements by name	a\$b

Operator Precedence

➤ $1+2*3$

[1] 7

Following the precedence rules, the program runs the multiplication first then moves onto addition.

➤ $(1+2)*3$

[1] 9

To have your program run the addition first, you enclose it with parenthesis.

Variables in R

- A variable is a placeholder in computer memory used to store data.
 - Unlike other languages (e.g., C, C++), R does not require variables to be declared for use. Any form of values can be assigned to a variable in R.
- When the variable appear elsewhere in the program, the latest value stored in that variable is used.
- Variable assignment in R:
 - `x <- 2` ## If we printed x now, R will print 2.
 - `y = 5`
 - `x <- y` ## What would R print out if we print(x) now?
 - `3 -> z`
 - `a <- b <- 7`

Naming Variables

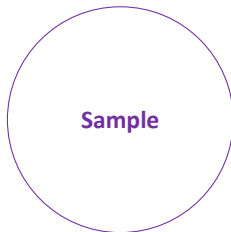
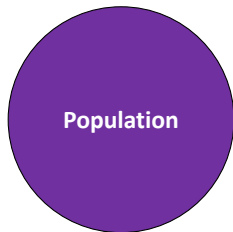
- Proper way to name a variable:
 - Any combination of alphanumeric characters (i.e., letters and numerals) along with periods (.) and underscores (_).
 - Cannot start with a number or an underscore.
 - Case sensitive (e.g., FOO, Foo, and foo are three different variables).

Good names	Names that cause errors
a	1trial
b	\$
FOO	^mean
my_var	2nd
.day	!bad

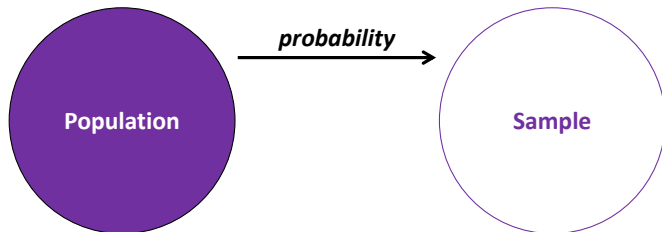
- ↪ Use variable names that clearly present the data stored in it (e.g., purchase.freq, price, quantity).

5/ Inferential Statistics Redux

Learning about Populations

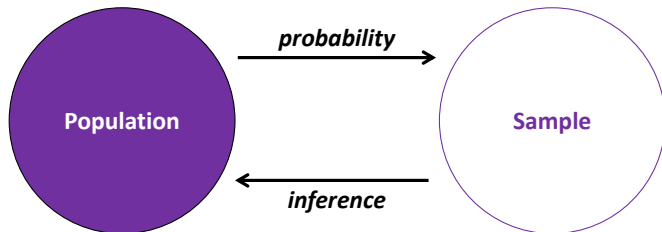


Learning about Populations



- **Probability:** formalizes the uncertainty about how our data came to be.
 - $\rightsquigarrow$ data generation process (DGP)

Learning about Populations



- **Probability:** formalizes the uncertainty about how our data came to be.
 - $\rightsquigarrow$ data generation process (DGP)
- **Inference:** learning about the population from a set of (sample) data.

Why Probability?

- Suppose men and women were paid the same on average, but there were variation from person to person.
 - How likely is the observed wage gap in this hypothetical world?

⇒ **Probability!**

- Ronald A. Fisher (1890–1962):
 - Statistical inference is a **thought experiment**.
 - Probability is the logic of these thought experiments.

The Milk-First versus Tea-First Debate

A very British debate rages on: what comes first, tea or milk?

A very British debate over the correct tea-drinking tradition reaches boiling point in impassioned letters to the editor in London newspaper, The Times.



What comes first - tea or milk? Source: Getty Images

- Your friend claims that (s)he can discern between milk teas that are prepared milk-first (vs. tea-first).
- Q: Can we trust her? $\rightsquigarrow$ Devise a test!
 - Prepare 8 cups of tea, 4 milk-first, 4 tea-first
 - Present cups to friend in a **random** order
 - Ask friend to pick which 4 of the 8 were milk-first

If We Knew the Truth..

- Friend picks out all 4 milk-first cups correctly!
- Statistical thought experiment: how often would she get all 4 correct **if she were guessing randomly?**
 - Only one way to choose all 4 correct cups.
 - But 70 ways of choosing 4 cups among 8 (i.e., 8C_4).
 - Choosing at random $\approx$ picking each of these 70 with equal probability.
- Chances of guessing all four correctly is $\frac{1}{70} \approx 0.014$ or 1.4%
- The guessing hypothesis is likely implausible w/ our p-value at 0.014!

Random Variables?

- Probability so far is about **events** and **outcomes**.
- Q: How do we connect it back to our data?

Random variable (r.v.):

r.v.s assign a numeric value to each outcome in the sample space.

⇒ **numeric representation** of uncertain/random events & outcomes.

Examples

- Random trials: tossing a coin 3 times.
 - One possible outcome is **HTH**.
 - but not a random variable because it's not numeric.
- Random variable: X = number of heads in the 3 tosses.
 - **HTH** $\rightsquigarrow X = 2$.
- We may formulate other r.v.s:
 - Y = number of tails
 - $Z = 1$ if any of the 3 flips are heads.

Types of Random Variables

- **Discrete r.v.:** X can take on a finite (or countably infinite) number of values.
 - Number of tails in n coin flips.
 - Number of deaths in a natural disaster.
 - Number of purchase conversions post advertising exposure.
- **Continuous r.v.:** X can take on any real value (usually w/in an interval).
 - GDP per capita (i.e., average income) in a country.
 - Amount of time spent on a website.
 - Share of population that favor Tesla.

Randomness and Probability Distribution

- How are r.v.s **random**?
 - Easiest way to think about the randomness and distributions: sampling.
- Randomly select 1 person from Amazon's customer base.
- Let $X = 1$ be if the person has subscribed for Amazon Prime, $X = 0$ otherwise.
 - $\mathbb{P}(X = 1)$ = the share of people with Prime membership in the population.
 - $\mathbb{P}(X = 0)$ = the share of people without the membership in the population.
- The **probability distribution** of a r.v. gives the probability of all of the possible values of the r.v.
 - Our Amazon example (r.v.s representing Amazon subscription) would give us a Bernoulli distribution with 2 possible outcomes.

Summarizing Distributions

- Our Amazon example had only two possible outcomes.
 - What about a r.v. W , wage of a randomly selected individual?
 - Problem: probability distributions can involve complex formulas that are hard to work with.
- Summaries of probability distributions:
 1. **Central tendency:** where the center of the distribution is.
 2. **Spread:** how spread out the distribution is around the center.
- Lots and lots of sample data about the r.v. W (i.e., wage):

```
> head(gapminder::gapminder$gdpPerCap, n = 216)
[1] 779.4453 820.8530 853.1007 836.1971 739.9811 786.1134 978.0114 852.3959 649.3414 635.3414 726.7341 974.5803
[13] 1601.0561 1942.2842 2312.8890 2760.1969 3313.4222 3533.0039 3630.8807 3738.9327 2497.4379 3193.0546 4604.2117 5937.0295
[25] 2449.0082 3013.9760 2550.8169 3246.9918 4182.6638 4910.4168 5745.1602 5681.3585 5023.2166 4797.2951 5288.0404 6223.3675
[37] 3520.6103 3827.9405 4269.2767 5522.7764 5473.2880 3008.6474 2756.9537 2430.2083 2627.8457 2277.1409 2773.2873 4797.2313
[49] 5911.3151 6856.8562 7133.1660 8052.9530 9443.0385 10079.0267 8997.8974 9139.6714 9308.4187 10967.2820 8797.6407 12779.3796
[61] 10039.5956 10949.6496 12217.2269 14526.1246 16788.6295 18334.1975 19477.0093 21888.8890 23424.7668 26997.9366 30687.7547 34435.3674
[73] 6137.0765 8842.5980 10750.7211 12834.6024 16661.6256 19749.4223 21597.0836 23687.8261 27042.0187 29095.9207 32417.6077 36126.4927
[85] 9867.0848 11635.7995 12753.2751 14804.6727 18268.6584 19340.1020 19211.1473 18524.0241 19035.5792 20292.0168 23403.5593 29796.0483
[97] 684.2442 661.6375 686.3416 721.1861 630.2336 659.8772 676.9819 751.9794 837.8102 972.7700 1136.3904 1391.2538
[109] 8343.1051 9714.9606 10991.2068 13149.0412 16672.1436 19117.9745 20979.8459 22525.5631 25575.5707 27561.1966 30485.8838 33692.6051
[121] 1062.7522 959.6011 949.4991 1035.8314 1085.7969 1029.1613 1277.8976 1225.8560 1191.2077 1232.9753 1372.8779 1441.2849
[133] 2677.3263 2127.6863 2180.9725 2586.8861 2980.3313 3548.0978 3156.5105 2753.6915 2961.6997 3326.1432 3413.2627 3822.1371
[145] 973.5332 1353.9892 1709.6837 2172.3524 2860.1698 3528.4813 4126.6132 4314.1148 2546.7814 4766.3559 6018.9752 7446.2988
[157] 851.2411 918.2325 983.6540 1214.7093 2263.6111 3214.8578 4551.1421 6205.8839 7954.1116 8647.1423 11003.6051 12569.8518
[169] 2108.9444 2487.3660 3336.5858 3429.8644 4985.7115 6660.1187 7030.8359 7807.0958 6950.2830 7957.9808 8131.2128 9065.8008
[181] 2444.2866 3008.6707 4254.3378 5577.0028 6597.4944 7612.2404 8224.1916 8239.8548 6302.6234 5970.3888 7696.7777 10680.7928
[193] 543.2552 617.1835 722.5120 794.8266 854.7360 743.3870 807.1986 912.0631 931.7528 946.2950 1037.6452 1217.0330
[205] 339.2965 379.5646 355.2032 412.9775 464.0995 556.1033 559.6032 621.8188 631.6999 463.1151 446.4035 430.0707
```

Central Tendency: Center of the Data

- “Center”?: typical/average value.
- **Mean**: sum of the values divided by the number of observations

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- **Median**: middle point in an ordered set of numbers.

$$\text{median} = \begin{cases} \text{middle value} & \text{if number of entries is odd} \\ \frac{\text{sum of two middle values}}{2} & \text{if number of entities is even} \end{cases}$$

- In **R**: `mean()` and `median()`.

Mean vs Median

- Median is more robust to **outliers**:
 - Example 1: data = $\{0,1,2,3,5\}$. Mean? Median?
 - Example 2: data = $\{0,1,2,3,100\}$. Mean? Median?
- Q: What does Elon Musk have to do with the mean & median income?

Which Distribution Would You Prefer?

- Lottery where we randomly draw one value from A or B:



- Risk seeking? Risk averse?
- They have the same mean, so why do we care about the difference?

Spread!

Spread of the Data

- Are the values of the variable close to the center (or not)?
- **Range:** $[\min(X), \max(X)]$
- **Quantile** (or percentile): divide data into equal sized groups.
 - 25th percentile = lower quantile (25% of the data below this value)
 - 50th percentile = median (50% of the data below this value)
 - 75th percentile = upper quantile (75% of the data below this value)
- **Inter-quantile Range (IQR):** A measure of variability
 - How spread out is the middle half of the data?
 - Is most of the data really close to the median or are the values spread out?
- **R functions:** `range()`, `summary()`, `IQR()`

Standard Deviation

- Standard deviation: On average, how far away are data points from the mean?

$$\text{standard deviation} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

- Steps:
 1. Subtract each data point by the mean.
 2. Square each resulting difference.
 3. Take the sum of these values.
 4. Divide by $n - 1$ (or n , doesn't matter much—Bessel's correction)
 5. Take the square root.
- Variance = standard deviation²
- Q: Why not just take the average deviations from mean without squaring? (or instead, using absolute values?)
 - $\rightsquigarrow$ Post your thoughts on Canvas!

Have a Great Weekend! :)

Contact Information:

jaewon.yoo@iss.nthu.edu.tw

<https://j1yoo.github.io/>



Appendix

Human Language vs. Computer Language

The Business Analytics (BA) module is intended for those interested in championing the use of data-driven decision making. The elective BA courses cover a range of topics used in industry by data scientists.

The ISS BA module is unique in its emphasis on **humane and socially responsible analytics**. Learn how to apply analytics to benefit consumers, employees, society, and the environment. The program also emphasizes data-driven **service innovation**: helping discover needs and aid service design using analytics.



```
0001010010110101010101111110110101011010
10110101101011010110101010111010111011101000
011011110011010111001110010100101001010110
10000110111100110101110011100101001010010101
00011010111001110010100101001010100011010111
00111001010010100101010001101011100111001010
01010010101000110101110011100101001010010101
00010100101001010100011010111001110010100101
00101010001101011100111001010010100100010100
10100101010001101011100111001010010111100101
00101001010100010100101001010100011010110...
```

Programming?

- **Programming** is a process of '*translating*' human language into a computer language which involves organizing formulas or tasks, setting the order of the steps.
 - Programming **languages** like R are tools that enable communication b/w humans and computers to solve a given logical problem.
 - **Coding** refers to writing commands to perform a specific task.

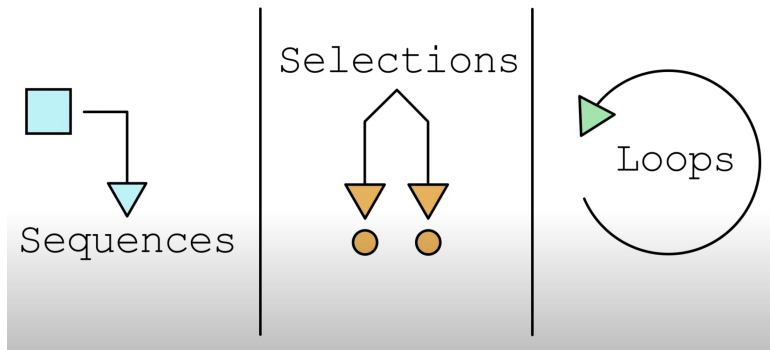
1. Choose any number and add 2 to it.
2. Multiply the result by 3.
3. Subtract 6 from the answer.
4. Divide what you get by 3.



```
x <- 10; x <- x + 2  
x <- x * 3  
x <- x - 6  
x <- x / 3
```

Programming Structures as Building Blocks

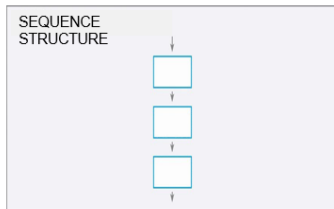
- Forms the basis of any advanced programs and algorithms:



Credit: <https://edu.gcfglobal.org/en/computer-science/sequences-selections-and-loops/1/#>

Building Block #1: Sequences

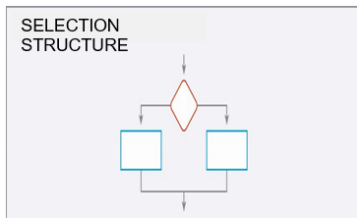
- In a sequence structure, each step/action is **executed in a specified order**.
 - Task 1 is performed, then Task 2, then Task 3, etc., until all of the tasks in the sequence have been executed.



```
1 x <- scan()  
2 x <- x*10  
3 print(x)
```

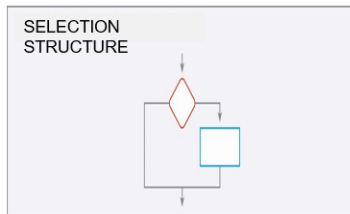
Building Block #2: Selections

- Instead of following a specific order of tasks, they **evaluate a question** in order to figure out which path to take next.
 - Selections are also referred to as **decision** structures.
 - The decision structure ends in the same 'node' regardless of which path is taken.



```
1  x <- scan()
2  if (x > 0)
3      x <- x * 10
4  else
5      x <- x / 10
```

Selection Structures May Skip Executing Task(s)!

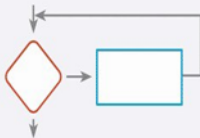


```
1 x <- scan()
2 if (x > 0)
3     x <- x * 10
4 print(x)
```


Building Blocks #3: Loops

- Evaluate the same question over and over and over again, until a certain task is complete (exits the loop).
 - Also referred to as **repetitions** or **iterations**.

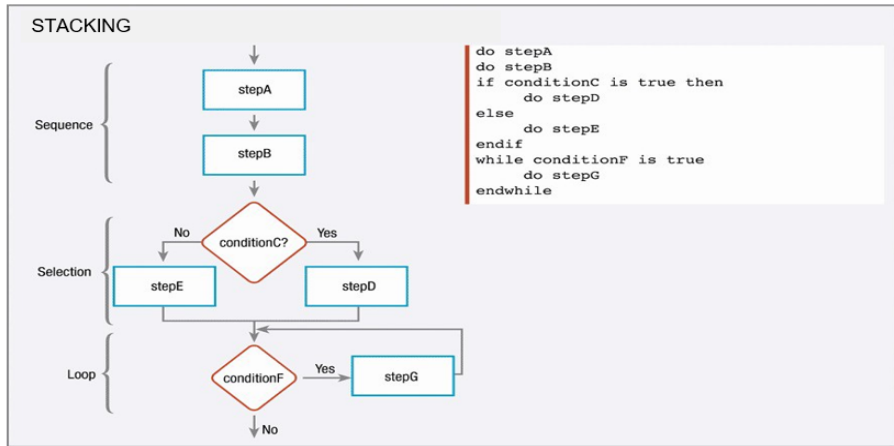
LOOP
STRUCTURE



```
1 x <- 1
2 while(x <=10) x<-x+4
3 print(x)
```

How We Put Them Together: Stacking

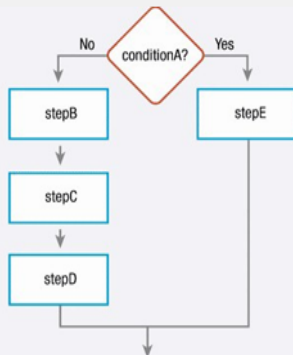
- Attaching a structure to the end of another structure is called stacking of programming structures:



Combining the Structures: Nesting

- Or, enclose a structure within another structure:

NESTING



```
if conditionA is true then
    do stepE
else
    do stepB
    do stepC
    do stepD
endif
```